LiveRecall: A Revision-Aware Live Caption Distribution System

CS3103 Assignment 4 | Group 11
Damian Liew Cho Xiang (A0302976X), Dylan Tang Zhi Yew (A0307803J),
Don Koo (A0307817Y), Leonard Lim Jia Hui (A0307813H)

1. Architecture and Protocol

LiveRecall streams lecture captions from one source to many browser clients over WebSocket. Each caption segment arrives as a sequence of revisions. Earlier revisions are provisional and may be replaced, and the last is final and permanent. The server (solution_server.py and the liverecall/ package) is a single asyncio process built on the websockets library, and the client is a static page in client/ (Figure 1).

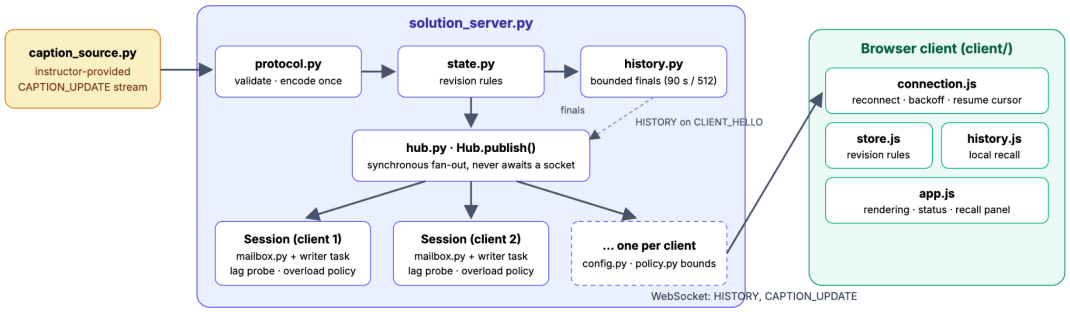


Figure 1. Architecture. Accepted captions pass through validation in protocol.py, revision state in state.py and the bounded history, then reaches one independent session (mailbox and writer task) per client to handle caption transmission.

Protocol. The source sends SOURCE_HELLO, then CAPTION_UPDATE messages (segment, revision, source_ts, final, text), then SOURCE_DONE. A client sends CLIENT_HELLO with its last finalised segment and revision and the history window it wants. The server replies with HISTORY (the finals the client is missing, oldest first) followed by live CAPTION_UPDATEs. The first message decides a connection's role. A malformed or missing first message (5s timeout), or an invalid client cursor, is closed with code 1008, so bad input doesn't crash the server.

Pipeline. For each source message the server will do the following:

1. Validates the message and removes unknown fields (protocol.py)
2. Re-encodes it **once** as compact JSON, so that it is smaller than the source's encoding and this "wire" is reused for every client and every HISTORY message so that there's no recomputation of JSON.
3. Applies the revision rules via state.py
4. Records final captions in the bounded history via history.py
5. Calls Hub.publish(), which places the caption in each connected client's mailbox

Hub.publish() is synchronous so it will only place the caption in each client's mailbox and wakes that client's writer task, and never awaits a socket such that a slow client will not delay transmission.This is unlike the starter kit given where each send has an await which means slow clients can hog the transmission.

2. Revision and Recovery Semantics

Revision rules. The server (state.py) and the browser (store.js) apply the same caption revision rules. A caption update is accepted only if its segment is new, or if the revision is strictly newer and the stored revision is not final. Old or repeated revisions are rejected, and so a finalised caption will not be overwritten. Both sides will also keep a bounded state where the server prunes the revision state to the history window, and the browser keeps a maximum of 512 segments. Both server and client also keep track of a minimum retained segment and reject anything below it, so an evicted segment will not reappear.

Bounded history. FinalHistory stores finals only, bound to 90s and 512 items. Maintains 2 lists sorted by timestamp and by segment to make window selection O(log n + k). Encoded HISTORY replies are cached in an LRU cache (64 entries, 8 MiB) that is cleared on every new final, so a reconnect burst

shares a cached encoded reply.

Resume and late join.

- **Resume:** the server returns every retained final after the client's disconnect, from the full 60 s window. It will ignore a smaller requested time window here to prevent missing finals.
- **Late join:** the client sends an initial message with segment 0, revision 0 and receives the last history_seconds of finals (at most 60).
- In both cases the current provisional caption is excluded. The client may already hold it, and a resend would count as a duplicate.

Atomic handover. The server builds the history snapshot and registers the client with the hub in one synchronous step, with no await in between. Because the event loop is single-threaded, no caption can be published between the snapshot and the start of live delivery. This ensures that every final arrives only once either in HISTORY or live.

Preventing duplicate finals. Three independent mechanisms enforce this:

1. HISTORY excludes everything at or before the client's current state.
2. Each mailbox drops provisionals for segments it has already finalised.
3. The browser treats finals as permanent.

Every run in Section 4 recorded 0 duplicate, 0 stale and 0 non-final HISTORY items.

Browser client.

- connection.js reconnects with capped exponential backoff (0.5 s rising to 5 s) plus a random jitter, detects half-open links when 30s pass without receiving a frame, and shows Connected, Reconnecting or Offline.
- history.js keeps 90 s of finals, so "What did I miss?" (15, 30 or 60 s; default 30 s) is answered locally without a server request.
- app.js updates one row per segment in place: [PROVISIONAL] rows are grey italic, [FINAL] rows are plain, and recovered rows are tagged "Recovered". It repaints at most once per animation frame and caps the page at 300 rows.

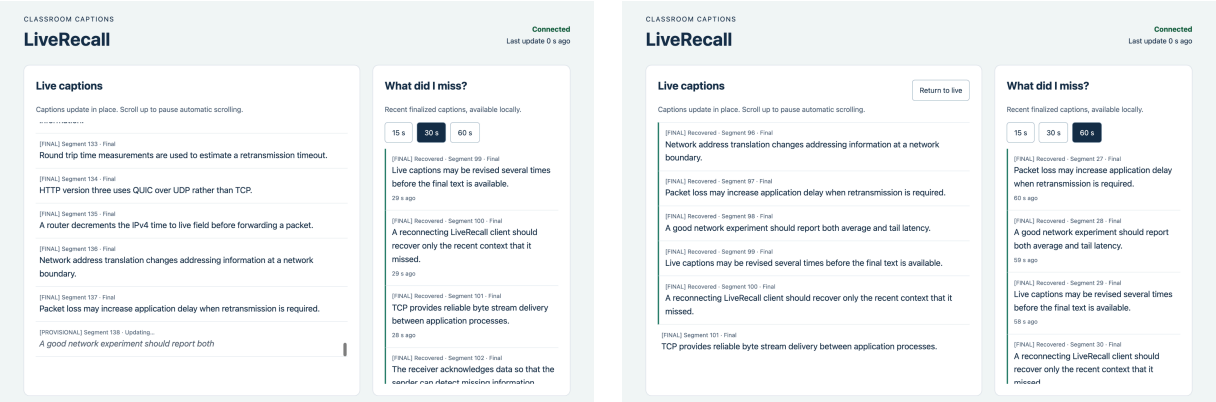


Figure 2. (a) Live view: Segment 138 is still provisional while earlier segments are final; the recall panel serves the last 30s locally. (b) After a disconnection, missed finals (96–100) arrive in HISTORY marked "Recovered", then live delivery resumes at 101 with no gap or duplicate.

3. Backpressure and Overload Policy

Why transport backpressure is insufficient. TCP backpressure reaches the server only when the client's buffers are full. A slow reader has two such buffers: its kernel receive buffer and its WebSocket library's 16-frame queue. At about 170 bytes per caption, they can hold minutes of stale captions before filling, so by the time the server is told to slow down, the reader is already far behind on stale text. We confirmed this on the starter server under T3 (10% of readers at 500 ms per message): the server never received backpressure, slow readers' P95 caption age reached 18.7 s, and final completeness fell to 95.3% (local measurement).

i. **Semantic mailbox.** Our server therefore holds each client's backlog itself, where outdated captions can still be discarded. For every connected client, the server keeps a separate mailbox (mailbox.py) of

captions waiting to be sent, and a separate writer task that sends from it. Because each client is served independently, a slow client can delay only itself. In the starter, the server sent to all clients in one loop, so one slow client delayed everyone. The mailbox has two parts:

- **Finals:** an ordered queue that is never coalesced or dropped.
- **Provisionals:** one slot per unfinished segment. A newer revision overwrites an unsent one, and a final removes it.

Finals are sent first. A client that keeps up therefore receives **every** revision, keeping full update coverage under normal conditions. A client that falls behind receives all finals plus only the newest provisional, and converges immediately.

ii. **Lag detection.** Each writer sends WebSocket pings between its data frames. A peer answers a ping only after reading every frame before it, so a pong acknowledges all earlier frames. Two rules use this:

- At most 32 frames may be unacknowledged, with a probe every 1 s or every 16 frames. Once lag is detected, the window drops to 1 frame until the client catches up. This bounds the backlog even during bursts.
- A pong later than 1 s counts as a **strike**.

State	Trigger	Delivery
Healthy	probes answered promptly	every revision, immediately
Stuck	probe overdue or window full	nothing sent; finals queue, provisionals coalesce
Catch-up	pong arrives	all held finals in one HISTORY frame
Degraded	after a strike, hold = 10 × stuck time, doubling per strike	finals as produced; provisionals at most 1 per second
Batched	estimated read time × finals rate ≥ 1	finals grouped, one frame per 1.5 read times

Table 1. Per-client overload states. A saturated reference client answers a probe only after draining its queue from 16 frames to 4, so its read time ≈ stuck time ÷ 12. When an impairment ends, the hold expires and the client returns to full revision delivery.

iii. **Bandwidth and bounds.** A slow network causes the same problem as a slow reader: captions pile up in buffers the server can no longer edit, this time on the server's side between the writer and the network. We keep those buffers small: 1 KiB for Python's write buffer and 1 KiB of unsent data in the kernel (`TCP_NOTSENT_LOWAT`). On a congested link, `send()` therefore pauses early and the backlog stays in the mailbox, where outdated provisionals are still replaced and finals still go first. Pings queued behind the slow data also arrive late, so congested clients enter the degraded state automatically.

Every buffer has a hard limit, so memory can never grow without bound:

- history: the last 90 s and at most 512 finals;
- per client: 512 pending finals, 1,024 pending updates, one provisional per segment, and 32 unacknowledged frames.

A client that exceeds a limit is disconnected with code 1013 ("try again later"). It then reconnects and receives everything it missed in one `HISTORY` message, so memory stays bounded without losing any final.

At startup the server raises the operating system's open-connection limit and accepts up to 4,096 pending connection requests, so a mass reconnect is not refused. Per-message deflate is enabled, so more captions fit through a constrained link, at a small CPU cost per connection. All of these limits can be changed through validated `LR_*` environment variables.

4. Experiments and Results

Method. The unmodified instructor harness was run on an Ubuntu 22.04.5 VM (Python 3.10.12, websockets 16.1.1):

- T1: 20 clients, 20 s.
- T2: 100 clients, 40 s.
- T3: 100 clients, 60 s, with 10% slow readers (500 ms per message).
- T4: 100 clients, 60 s, with 20% disconnecting for 5 s at 15 s.

- T5: T2 runs under netem with 120 ± 30 ms delay and 3% loss.

Each test was run three times with a server restart between runs, and the medians of the per-run metrics are reported. Freshness uses the 500 ms threshold. Raw logs are kept in experiments/20261102/.

Test	P50 (ms)	P95 (ms)	P99 (ms)	Fresh (%)	Final completeness (%)	Fresh finals (%)
T1	1.77	2.79	4.44	100.00	100.00	100.00
T2	5.49	9.54	10.93	100.00	100.00	100.00
T3	4.70	8.69	4,992	98.09	100.00	92.89
T4	4.79	8.24	9.13	100.00	100.00	98.42
T5	248.13	664.36	975.18	93.08	100.00	88.27

Table 2. Public tests, medians of three runs. All 15 runs recorded 0 duplicate, 0 stale, 0 invalid-HISTORY and 0 connection-error events.

- **T1, T2:** 100% update coverage. Going from 20 to 100 clients raised P99 only from 4.4 to 10.9 ms.
- **T3:** 100% completeness. Normal readers were unaffected (P95 8.19 ms, versus 9.54 ms in T2) and each received all 228 revisions. The 4,992 ms P99 comes only from slow readers (their P95 was 9.6 s), which skip obsolete provisionals by design. The analyser's coverage field (91.8%) includes those slow readers.
- **T4:** all disconnected clients recovered every final, with a recovery P95 of 4.88 ms and no duplicates.
- **T5:** completeness stayed at 100% while P95 rose to 664 ms. TCP retransmits lost segments and delivers bytes in order, so one loss delays every caption queued behind it (head-of-line blocking). Loss therefore shows up as tail *delay*, not as missing captions (Appendix Figure A1).

Before/after optimisation. T3 was re-run twice, changing only LR_DELIVERY. Both runs used the same test configurations. The baseline is a bounded FIFO with the same per-client queues and limits, but no coalescing, probes or batching.

T3, median of 3	Final Caption Completeness	P95	P99	Slow-reader P95	Normal-reader P95	Fresh finals
Bounded FIFO	95.13%	2,254 ms	24,049 ms	28,121 ms	8.46 ms	90.00%
Semantic (ours)	**100.00%**	**12 ms**	**5,589 ms**	**9,008 ms**	10.40 ms	**94.34%**

Table 3. Before/after comparison. Per-client queues alone already isolate normal readers; the semantic policy is what keeps slow readers complete. Recorded on an earlier revision with 4 KiB writes and compression off; absolute latencies are not comparable with Table 2.

Under FIFO, slow readers replay every obsolete revision and are still missing finals when the test ends. Semantic delivery discards obsolete provisionals and batches held finals into catch-up frames. This restores 100% completeness and cuts slow-reader P95 at a cost of about 2 ms of normal-reader P95. Additional recovery-at-scale and slow-reader stress results are in Appendix A.

5. Failures, Trade-offs and Limitations

- **Backpressure did not fire in the initial slow-reader experiment.** Our first design reacted only when a send was blocked, which never happens for slow readers. This led to the ping-as-acknowledgement detector.
- **Bursts defeated time-only detection.** About 80 frames reached a slow reader before a 1 s probe noticed. Adding the 32-frame window raised burst-test completeness from 90.5% to 99.7%.
- **A fixed 4 seconds hold oscillated** between catching up and falling behind. Making the hold proportional to the time stuck fixed this.
- **Readers slower than the finals rate** reached only 85.9% completeness until finals were batched (100% after).
- **Freshness vs completeness.** Finals recovered through catch-up are complete but often older than 500 ms (T3 fresh finals: 92.9%). We chose completeness, because a missing final is a correctness failure. Closing clients on overflow (code 1013) bounds memory but costs a reconnect, and recovery is limited to the 90s history.

- **Limitations.**
    - A slow reader is detectable only after its 16-frame client queue fills, about 8s at 500 ms per read. This is the residual slow-reader tail.
    - The read-rate model is calibrated to the reference client.
    - The per-socket buffer sizes have not been swept under tight shared bandwidth caps.
    - The state is in memory only.
    - The public results come from deterministic single-machine workloads and do not establish hidden-test performance.

6. Future Work / FYP Direction

Adaptive per-client scheduling driven by measured lag and estimated bandwidth could restore full revision delivery sooner after an impairment ends. Optional client acknowledgements, or QUIC/WebTransport streams, would remove both the detection floor and TCP head-of-line blocking.

7. Bonus Feature: Real-Lecture ASR Integration

Same-system integration. The ASR pipeline operates with zero modifications to the core server (`solution_server.py`) or browser client (`client/app.js`). All ASR packages (`vosk`, `imageio-ffmpeg`, `websockets`) are isolated in `bonus_asr/requirements-asr.txt` and run in an independent virtual environment (`.venv-asr/`). The ASR streamer (`asr_source.py`) connects to `solution_server.py` over WebSocket, performing the standard `SOURCE_HELLO` handshake and streaming `CAPTION_UPDATE` frames with identical payload semantics.

End-to-end pipeline.

1. **Audio Ingestion:** A standalone, unprivileged `ffmpeg` process extracts and streams 16 kHz, 16-bit mono raw PCM audio in 200 ms chunks at a calibrated 1.0× real-time rate.
2. **Streaming Recognition:** A local Vosk Kaldi engine (`vosk-model-small-en-us-0.15`, 40 MB) decodes acoustic features chunk by chunk without requiring external network APIs or pre-generated transcripts.
3. **Revision Mapping:** As the lecturer vocalizes words within an utterance, Kaldi's partial hypotheses are dispatched immediately as provisional updates (`final=false`) with monotonically increasing `revision` numbers for the active `segment`.
4. **Finalization & History:** When natural pause/VAD silence is detected, the engine commits the sentence with `final=true`. `solution_server.py` archives the durable final in `FinalHistory`, while `asr_source.py` increments `segment` and resets `revision=0`. The browser client renders provisional revisions in italicized text in place, locks finalized lines into the live transcript, and serves them immediately to the "What did I miss?" recall cache.

Timing and integration limitations

- **Acoustic Latency vs Accuracy:** Unlike `caption_source.py` which emits pre-scripted revisions instantaneously, real ASR acoustic models require future context frames (typically 200–500 ms of right-context) and beam-search decoding before phoneme hypotheses stabilize. This introduces an unavoidable physical floor of ~300–600 ms between vocalization and first provisional emission.
- **VAD Boundary Ambiguity:** Live speech lacks punctuation. Relying on acoustic silence for segment finalization is imperfect: mid-sentence hesitations risk premature finalization (splitting a clause across two segments), whereas fast continuous speech delays `final=true`, holding text provisional and temporarily delaying history snapshot caching.
- **Host Resource Contention:** Local neural matrix computations consume significant CPU time. Running the ASR engine on the same evaluation host as the WebSocket distribution server can starve Python's single-threaded `asyncio` event loop, inducing event-loop jitter that inflates measured WebSocket tail latency (P95/P99) unless processes are pinned to separate cores.

References

[1] I. Fette and A. Melnikov, *The WebSocket Protocol*, RFC 6455, IETF, 2011.
[2] websockets v16 documentation: memory and buffers (max_queue, write_limit).
[3] Linux manual page tcp(7): TCP_NOTSENT_LOWAT.
[4] NUS School of Computing, *CS3103 Assignment 4 Mini-Project: LiveRecall*, 2026.

Appendix A. Additional Results

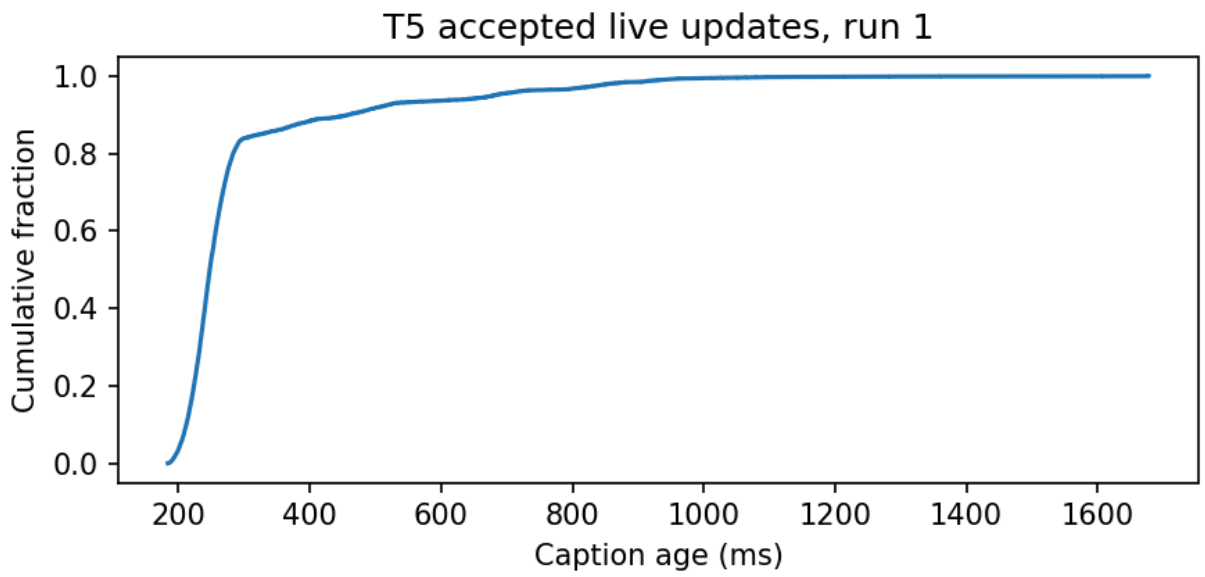


Figure A1. T5 caption-age CDF (run 1 of the earlier revision). The tail beyond the ~240 ms round trip reflects retransmission and head-of-line blocking.

Scenario	Workload	Recovery P50	Recovery P95	Recovered items	Final compl.
H5 disconnect storm	300 clients; 50% disconnect for 10 s	26.71 ms	28.08 ms	1,950	100.00%
H6 late joiners	200 clients; 30% join 15 s late	0.47 ms	1.08 ms	1,140	100.00%

Table A1. Recovery at scale (no impairment, medians of 3). All runs: 0 duplicate, 0 stale, 0 invalid-HISTORY, 0 connection errors. Cached HISTORY replies and the atomic handover keep a 150-client reconnect burst at a 28 ms P95.

Scenario (macOS laptop, medians of 3)	Starter	Ours
H2: 300 clients, 30% slow at 1000 ms — final completeness	77.50%	**100%**
H2 — normal-reader P95	7.0 s	**39 ms**
H7: 10× revision rate, 200 clients, 10% slow — final completeness	90.53%	**99.71%**

Table A2. Slow-reader stress tests. In the starter, one slow group stalled every client; per-client writers keep normal readers fast. Latencies are not comparable with Table 2 (different machine).

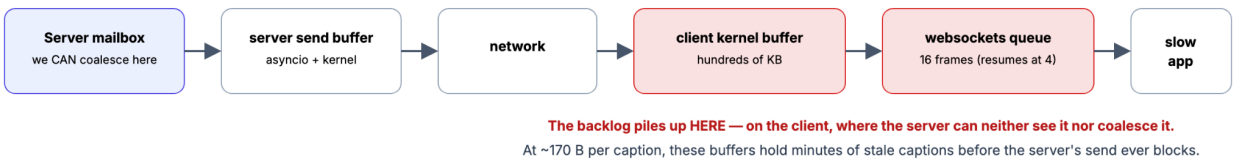


Figure A2. Where a slow reader's backlog accumulates: only data still in the server mailbox can be coalesced.

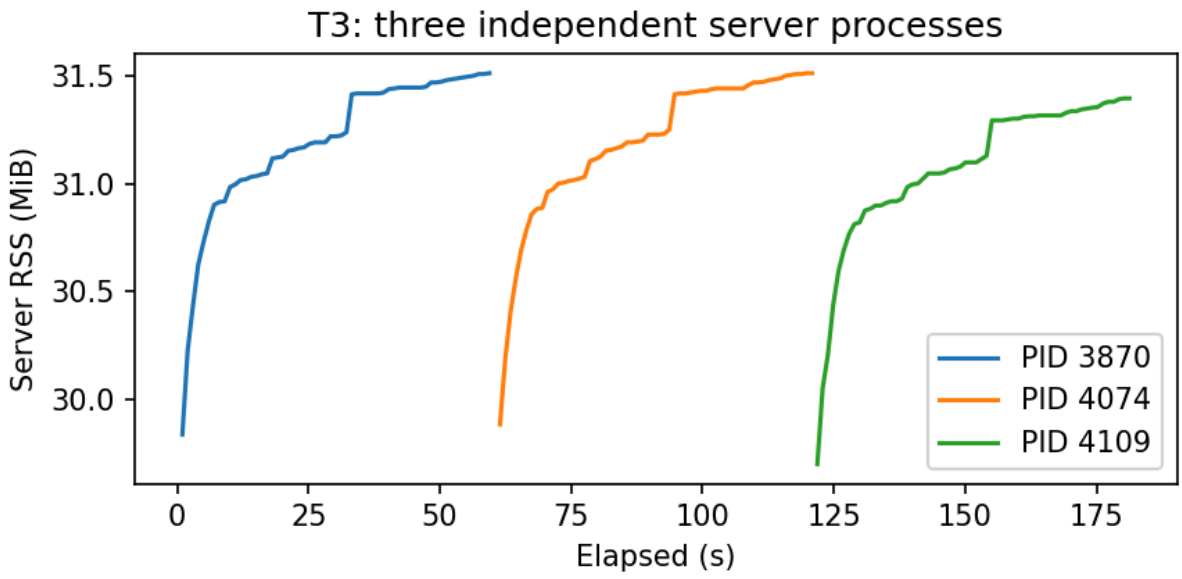


Figure A3. Server RSS during T3 stays at about 30-31.5 MiB with sustained slow readers.